

Architecture et Programmation Web

Séance 7 : Architecture MVC et Modularité

Nicolas Burny

10 avril 2026

ECAM

Organisation et Modularité

L'évolution d'une application

- Un seul fichier `app.py` (Séances 1-6).
- 500 lignes, 20 routes, 10 modèles.
- **Le problème :**
 - Difficile de s'y retrouver.
 - Imports circulaires fréquents.
 - Travail en équipe complexe sur un seul fichier.

La solution : Diviser pour régner

- **Architecture MVC** (Model-View-Controller)
 - Séparer la logique métier, l'accès aux données et l'affichage.
 - Facilite la maintenance et les tests unitaires.
- **Modularité via les Blueprints**
 - Découper l'application en modules autonomes (ex : `auth`, `blog`, `admin`).
 - Chaque module a ses propres routes et templates.
- **Application Factory Pattern**
 - Remplacer l'objet global `app` par une fonction `create_app()`.
 - Permet de configurer plusieurs instances (tests, développement, production).
 - Évite les problèmes d'imports circulaires.

Le Pattern MVC

Model-View-Controller (MVC)

Modèle : La Donnée

Représente les informations et la logique métier (Classes SQLAlchemy).

Vue : L’Affichage

Gère la présentation visuelle pour l'utilisateur (Templates Jinja2).

Contrôleur : Le Processeur

Reçoit les requêtes, interagit avec le modèle et sélectionne la vue (Routes Flask).

Cycle d'une requête en MVC

1. Le **Contrôleur** reçoit la requête HTTP.
2. Il demande des données au **Modèle**.
3. Le **Modèle** interroge la base et renvoie des objets Python.
4. Le **Contrôleur** passe ces objets à la **Vue**.
5. la **Vue** génère le HTML final.
6. Le **Contrôleur** renvoie la réponse au client.

Organisation en Packages

Qu'est-ce qu'un Package Python ?

- Un dossier contenant un fichier `__init__.py`.
- Permet d'importer le contenu via `import mon_package`.
- Le fichier `__init__.py` est exécuté dès que le package est importé.

```
/app
  __init__.py    # Création de l'application
  models.py      # Modèles de données
  /main          # Package principal
    routes.py
  /features      # Package "Fonctionnalités"
    routes.py
  /templates     # Vues
  /static        # Fichiers statiques
```

Application Factory

Le concept d'Application Factory

- Au lieu de créer une instance globale `app = Flask(__name__)`, on définit une fonction (souvent `create_app()`) qui instancie l'objet.
- Cette fonction configure l'application et enregistre les blueprints avant de la retourner.

Pourquoi une application factory ?

- **Imports circulaires** : Si `models.py` importe `app` et `app.py` importe `models`, rien ne marche.
- **Tests** : On veut pouvoir créer une application "vierge" pour chaque test.
- **Configuration** : Facile de passer d'une base de données SQLite à PostgreSQL selon le contexte.

Implémentation de create_app()

```
from flask import Flask
from .models import db

def create_app():
    app = Flask(__name__)
    app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'

    db.init_app(app)

    # Enregistrer les blueprints ici...

    return app
```

Résoudre les imports circulaires

- **Le Problème** : `app.py` importe `models.py` (pour initier la DB) et `models.py` importe `app` (pour décorer les routes ou accéder à la config).
- **La Solution Factory** :
 - Les extensions (`db = SQLAlchemy()`) sont créées sans application.
 - Elles sont liées dynamiquement via `db.init_app(app)` dans la factory.
 - On utilise des **Blueprints** pour définir les routes sans l'objet `app`.

Le Problème : Import Circulaire

app.py

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from models import User # Import pour db.create_all()

app = Flask(__name__)
db = SQLAlchemy(app)

db.create_all() # Nécessite d'avoir importé User
```

↑ app.py attend que models soit chargé.

models.py

```
from app import db # BLOQUE ICI

class User(db.Model):
    # ...
```

↑ models.py attend que app.py définisse db.

Blocage

Python ne peut pas terminer l'importation de l'un car il a besoin de l'autre immédiatement (Import Circulaire).

Exemple : Structure sans cycle

extensions.py (ou models.py)

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy() # Pas d'app ici !
```

routes.py

```
from flask import Blueprint
main = Blueprint('main', __name__)

@main.route('/')
def index():
    return "Plus besoin d'importer app !"
```

app.py (La Factory)

```
from flask import Flask
from .extensions import db
from .routes import main

def create_app():
    app = Flask(__name__)

    # On lie l'extension à l'instance
    db.init_app(app)

    # On enregistre les routes
    app.register_blueprint(main)

    return app
```

- **Le Problème** : Un objet app global reste chargé en mémoire et "pollue" l'état entre deux tests.
- **La Solution Factory** :
 - On appelle `create_app()` au début de chaque test pour une isolation totale.
 - Possibilité d'injecter une config spécifique (`TESTING=True`).
 - Usage d'une base SQLite **en mémoire** (`sqlite:///memory:`) pour la rapidité.

Tests : Le problème de l'objet global

test_app.py (Lancé via la commande pytest)

```
from app import app, db # Import de l'instance UNIQUE (Singleton)

# Pytest découvre et appelle ces fonctions l'une après l'autre
def test_creation_user():
    u = User(name="Alice")
    db.session.add(u)
    db.session.commit()
    assert User.query.count() == 1

def test_db_vide():
    # Ce test ÉCHOUÉ car Alice est restée en mémoire !
    # L'application et la DB n'ont pas été réinitialisées.
    assert User.query.count() == 0
```

Pollution d'état

Sans Factory, l'objet app est importé comme un **Singleton**. L'état de la base de données est partagé entre tous les tests, créant des effets de bord.

- **Le Problème** : Un objet app global reste chargé en mémoire et "pollue" l'état entre deux tests.
- **La Solution Factory** :
 - On appelle `create_app()` au début de chaque test pour une isolation totale.
 - Possibilité d'injecter une config spécifique (`TESTING=True`).
 - Usage d'une base SQLite **en mémoire** (`sqlite:///memory:`) pour la rapidité.

conftest.py (Fixture Pytest)

```
from app import create_app, db

@pytest.fixture
def db_client():
    # On crée une NOUVELLE instance pour CHAQUE test
    app = create_app(TestConfig)
    with app.app_context():
        db.create_all()
        yield app.test_client()
        db.drop_all() # Nettoyage total après le test
```

- Chaque test démarre avec une application "neuve".
- Isolation totale : plus d'effets de bord imprévus.

- **Le Besoin** : Changer les secrets et URLs de DB selon l'environnement.
- **La Solution Factory** :
 - `create_app(config_class)` prend la configuration en paramètre.
 - `app.config.from_object(config_class)`.
 - Switch facile : SQLite en Dev, PostgreSQL en Production, sans modifier le code source.

Problème : Configuration Figée

app.py (Configuration "en dur")

```
from flask import Flask

app = Flask(__name__)

# Obligé de modifier le code pour changer de base de données
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SECRET_KEY'] = 'dev-key-123'

# Comment faire pour utiliser une base de test ou de production ?
```

Rigidité

Le code n'est pas portable. On risque d'envoyer accidentellement des clés de développement ou des URLs de bases locales en production.

Solution : Classes de Configuration

```
class Config: # Base commune
    SECRET_KEY = 'super-secret'

class DevConfig(Config):
    SQLALCHEMY_DATABASE_URI = 'sqlite:///dev.db'

class ProdConfig(Config):
    SQLALCHEMY_DATABASE_URI = 'postgresql://user:password@remote-db/prod'

def create_app(config_class=DevConfig): # Injection de la config
    app = Flask(__name__)
    app.config.from_object(config_class)

    db.init_app(app)
    return app
```

Les Blueprints

C'est quoi un Blueprint ?

- Un "mini-serveur" Flask autonome.
- Il définit ses propres routes, templates et dossiers statiques.
- Il n'est actif que lorsqu'il est **enregistré** sur l'application principale.

Pourquoi utiliser des Blueprints ?

- **Modularité** : Découper une grosse application en composants logiques (ex : auth, blog, admin).
- **Organisation** : Éviter d'avoir un fichier `routes.py` de plusieurs milliers de lignes.
- **Préfixes d'URL** : Appliquer automatiquement un préfixe à toutes les routes d'un module (ex : `/api/v1/`).
- **Réutilisabilité** : Facilite le partage de fonctionnalités entre différents projets Flask.

Définir un Blueprint

Fichier app/main/routes.py :

```
from flask import Blueprint

main = Blueprint('main', __name__)

@main.route('/')
def home():
    return "Bienvenue !"
```

Enregistrer un Blueprint

Dans `app/__init__.py` :

```
def create_app():  
    # ... setup app ...  
  
    from app.main.routes import main  
    app.register_blueprint(main)  
  
    from app.features.routes import features  
    app.register_blueprint(features, url_prefix='/features')  
  
    return app
```

- **url_prefix** : Toutes les routes du blueprint features commenceront par /features/.
- **url_for** : On doit maintenant spécifier le nom du blueprint.
 - `url_for('main.home')`
 - `url_for('features.add')`

Note : pourquoi des imports locaux ?

Le problème : La dépendance mutuelle

`__init__.py` importe `routes.py` **ET** `routes.py` importe `db` depuis `__init__.py`.

- **Imports au sommet** : Créent un blocage car les fichiers s'attendent mutuellement.
- **Imports dans la fonction** : Diffèrent l'importation au moment où `create_app()` est exécutée.
- À ce stade, l'objet `db` est déjà créé et disponible pour les routes.

Exemple d'Import Local

app/__init__.py

```
from flask import Flask
from .models import db

def create_app():
    app = Flask(__name__)
    db.init_app(app)

    # IMPORT LOCAL
    # On attend que 'db' existe
    from .main.routes import main
    app.register_blueprint(main)

    return app
```

app/main/routes.py

```
from flask import Blueprint
# On importe 'db' de l'app
from app import db

main = Blueprint('main', __name__)

@main.route('/')
def home():
    # 'db' est déjà lié à l'app
    # grâce à init_app !
    return "Tout fonctionne !"
```

Pourquoi ça marche ?

Python n'exécute le code de `routes.py` que lorsque la fonction `create_app()` est appelée. À ce moment-là, l'objet `db` est déjà défini dans `__init__.py`, évitant de tomber sur un objet "en cours de création".

Organisation des Vues

Organisation recommandée pour éviter les collisions de noms entre blueprints :

- `templates/base.html` (Squelette global)
- `templates/main/index.html` (Vue du blueprint main)
- `templates/features/add.html` (Vue du blueprint features)

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}Mon App{% endblock %}</title>
</head>
<body>
    <nav>...</nav>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
</body>
</html>
```

```
{% extends "base.html" %}
```

```
{% block title %}Accueil{% endblock %}
```

```
{% block content %}
```

```
    <h1>Bienvenue</h1>
```

```
    <p>Ceci est le contenu de la page d'accueil.</p>
```

```
{% endblock %}
```

Appel depuis les routes

```
# app/main/routes.py
@main.route('/')
def index():
    return render_template('main/index.html')

# app/features/routes.py
@features.route('/add')
def add():
    return render_template('features/add.html')
```

Conclusion

- **Application Factory** : Centralisation de la création de l'application Flask pour une meilleure configuration et extensibilité.
- **Blueprints** : Découpage de l'application en modules logiques pour faciliter la maintenance et l'évolution.
- **Templates & Héritage** : Structuration des vues avec un layout de base (`base.html`) et des sous-répertoires par blueprint.

Questions ?